

Hash Table (Map , Dictionary , Key Value Pair)

Struct Item
{
 int Key ;
 string Name;
 double Salary;
}

Item I01 = {1 , "Ahmed" , 5000};
Item I02 = {3 , "Ali" , 15000};
Item I03 = {4 , "Mona" , 13000};
Item I04= {7 , "Sayed" , 3000};

GetItem (int Key)
{ // int HashValue = GetIndex (item.Key)
 /// Items[Hash Valye]
 return Items[GetIndex(item.Key)];
}

Fixed Time

HashValue Hash Function (Key)

int GetIndex(int Key)
{
 return Key % TableSize;
}
Range (0 : TableSize-1)

Insert (ITem item)
{
 Vector<item>
 Items[GetIndex(item.Key)] = item;
}

Collision : Two items
Produces Same Index

Best Case O(1) : Const Time
Worst Case Bucket Lenght

Chaining

Bucket

Hash Table

Item[] Items

Vector<List<Item>> : Chaining

Item I05 = {14 , "Sally" , 7000};

14
~~10~~

struct Employee
{
 string Key ;
 string Name ;
 double Salary ;
};

1. Avoid Collision
2. Distribute Values in
Table evenly
(will be easy if keys
random Numbers)

TableSize : 10037
unsigned int Hash (string Key , int TableSize)
{
 unsigned int HashValue = 0;

 for (char Ch : Key)
 HashValue += Ch ;

 retun HashValue % TableSize;
}

TableSize : 10037
unsigned int Hash (string Key , int TableSize)
{
 unsigned int HashValue = 0;

 for (char Ch : Key)
 HashValue = 37 * HashValue + Ch ;

 retun HashValue % TableSize;
}

TableSize : 10037
unsigned int Hash (string Key , int TableSize)
{
 unsigned int HashValue = 0;

 HashValue = Key[0] + 27*Key[1] + 27*27*Key[2] ;

 retun HashValue % TableSize;
}

Template <class KeyType , class ValueType>
struct MapEntry
{
 KeyType Key ;
 ValueType Value;

public :

 KeyType GetKey();
 ValueType GetValue();
}

Vector<List<MapEntry>> HashTable;

X

5MB

Hash Function
55OI44 ~~X~~
19YH25 ✓
Found Or Collision

Equals (X , Ar(1))

12AB56

56GH89

19YH25

63TR94

3

2

1

0

5 MB

Ar

Adjacency List

EG

LI

ALG

LI : 100

EG : 100

LI : 200

ALG : 200

Graph

Set of Verticed
Set of Edges

Adjacency Matrix
3 * 3

EG

LI

ALG

EG

LI

ALG

0

100

0

100

0

200

0

200

0